# Project - Low Level Design
# On
# Dockerized Marketing PHP Laravel Service

Course Name: Devops

***Institution Name:*** Medicaps University – Datagami Skill Based Course

*Student Name(s) & Enrolment Number(s):*

| Sr no | Student Name | Enrolment Number |
|---|---|---|
| 1. | Kavish Jha | EN22CS301494 |
| 2. | Kritika Purohit | EN22CS301527 |
| 3. | Kunal Patidar | EN22CS301538 |
| 4. | Jatin Patidar | EN22CS301458 |
| 5. | Javesh Khargonkar | EN22CS301460 |
| 6. | Krishna Pawar | EN22CS301524 |

*Group Name: Group 06D5*

*Project Number: DO-06*

*Industry Mentor Name:* Mr. Vaibhav

*University Mentor Name:* Prof. Ritesh Joshi

*Academic Year:* 2025 - 26

# 1. Introduction.

## 1.1. Scope of the document.

This document describes the Low Level Design (LLD) of the Dockerized Marketing Laravel Service.
It covers the internal structure, containerization process, and interaction between application components.

The scope includes:
- Docker-based containerization of Laravel application
- Creation of Docker images using Dockerfile
- Multi-container orchestration using Docker Compose
- Integration with MySQL database container
- Execution flow from source code to running application

## 1.2. Intended Audience

This document is intended for:
- Developers working on the project
- DevOps engineers responsible for deployment
- Faculty and evaluators reviewing the system design
- Beginners learning containerization and DevOps practices

## 1.3. System overview.

The system is a Dockerized web application built using the Laravel PHP framework.
The application is containerized using Docker and runs inside isolated containers.

The system consists of:
- Laravel Application (PHP-based backend)
- MySQL Database
- Docker for containerization
- Docker Compose for orchestration

The application runs on a local server and is accessible via a web browser.

# 2. System Design.

## 2.1. Application Design

The application follows a container-based architecture.

Main components:

1. Laravel Application Container

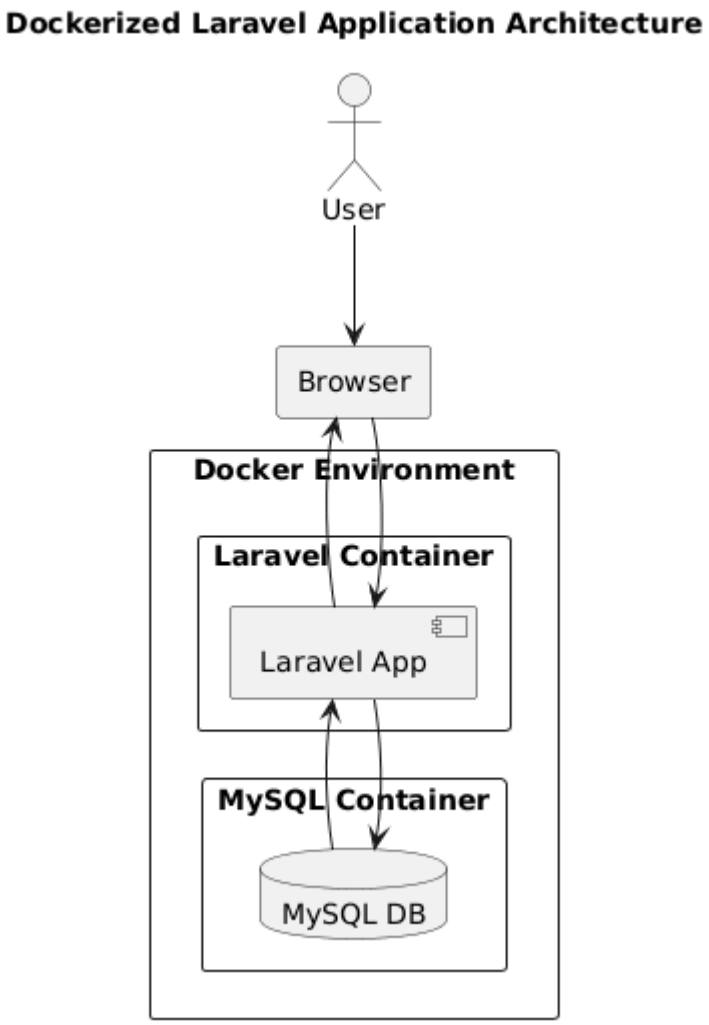
- Runs PHP and Laravel framework
- Handles business logic and HTTP requests

2. MySQL Database Container
   - Stores application data
   - Connected to Laravel via environment variables

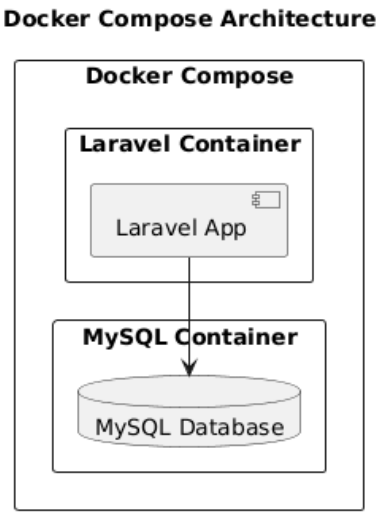
3. Dockerfile
   - Defines how the Docker image is built

4. Docker Compose
   - Manages multiple containers (App + Database)

Architecture Flow:
User → Browser → Laravel Container → MySQL Container



The following diagram illustrates the multi-container architecture using Docker Compose.



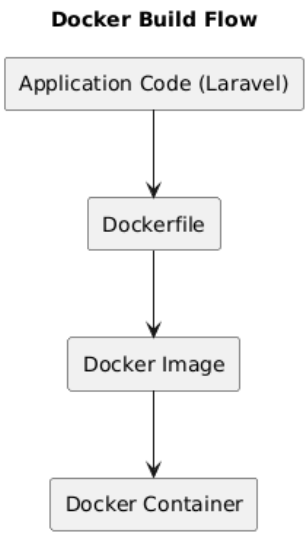

2.2. Process Flow.

The process flow of the system is as follows:

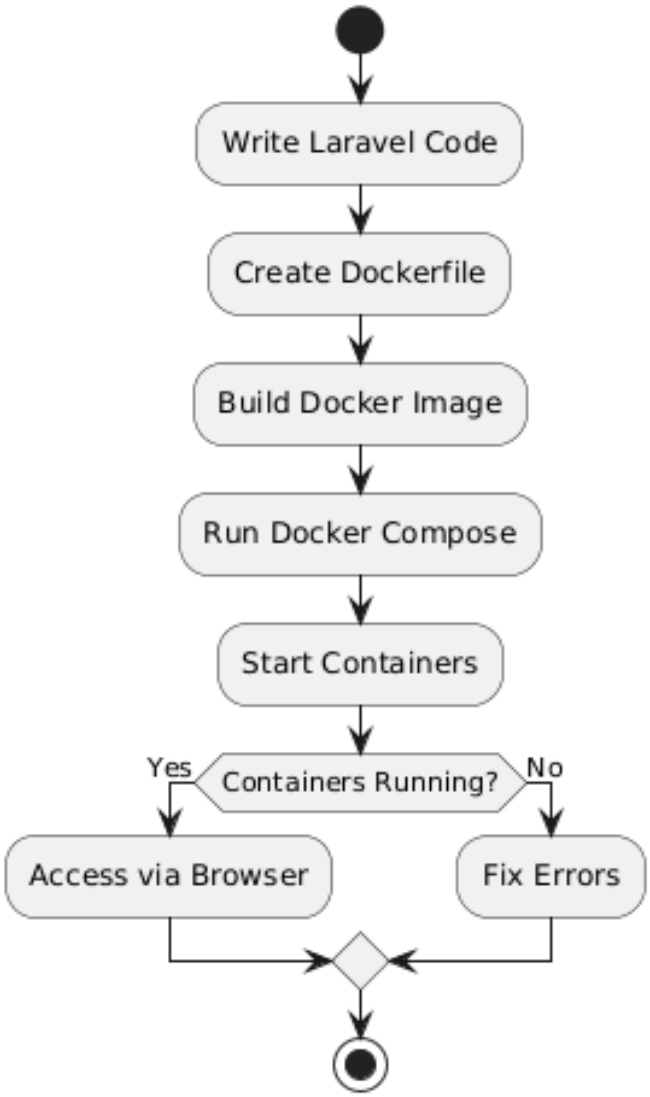
1. Developer writes Laravel application code
2. Dockerfile is used to build Docker Image
3. Docker Image is created using docker build command
4. Docker Compose starts containers
5. Laravel container connects to MySQL container
6. User accesses application via browser (localhost:8000)

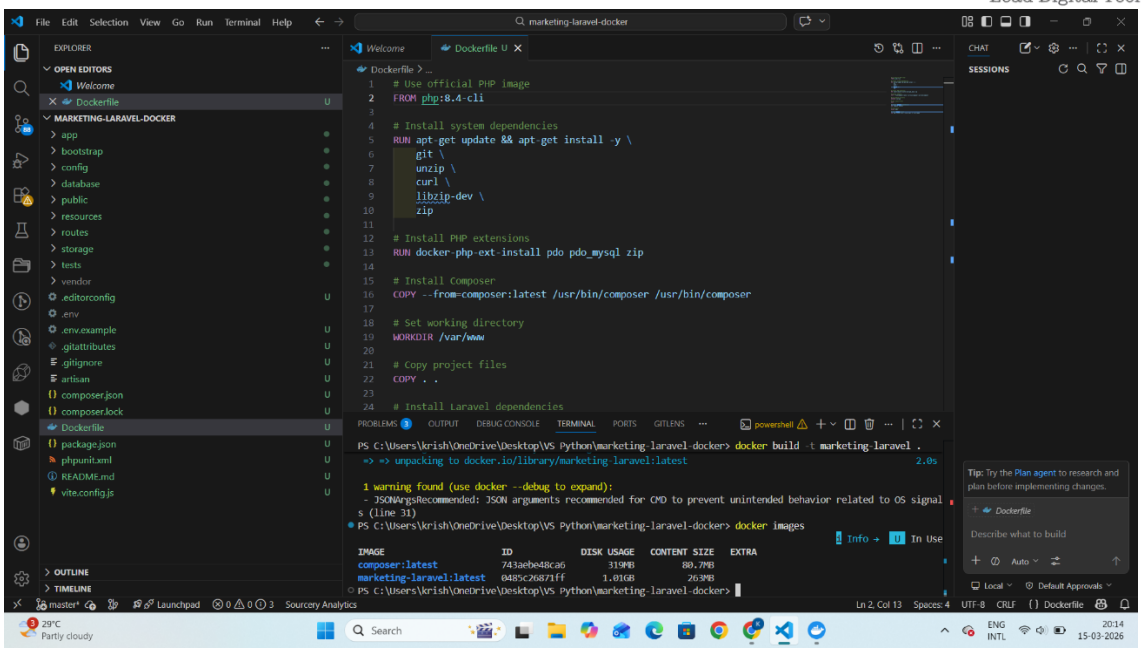
Flow Diagram:
Code → Dockerfile → Docker Image → Docker Container → Browser

The following diagram represents the transformation of application code into a running container using Docker.



**Process Flow - Dockerized Laravel**






➔ docker build -t marketing-laravel .
➔ docker images

Fig. - Docker Image is created using docker build command

## 2.3. Information Flow.

1. User sends request via browser
2. Request reaches Laravel application container
3. Laravel processes request
4. If required, Laravel queries MySQL database
5. MySQL returns data to Laravel
6. Laravel sends response back to browser

Flow:
User → Laravel → MySQL → Laravel → User

## 2.4. Components Design

1. Laravel Application
   - Framework: Laravel (PHP)
   - Handles routing, controllers, and views

2. Dockerfile

   - Base Image: php:8.4-cli
   - Installs dependencies
   - Copies application code
   - Runs Laravel server

3. Docker Image
   - Built from Dockerfile
   - Contains application + dependencies

4. Docker Container
   - Running instance of Docker Image
   - Executes Laravel application

5. MySQL Container
   - Database service
   - Stores application data

6. Docker Compose
   - Defines multi-container setup
   - Links Laravel and MySQL containers

## 2.5. Key Design Considerations

1. Containerization
   - Ensures consistent environment across systems

2. Portability
   - Application can run anywhere using Docker

3. Scalability
   - Can be extended to Kubernetes deployment

4. Isolation
   - Each service runs in its own container

5. Security
   - Dependencies are isolated inside containers

6. Maintainability
   - Easy to update and manage services independently

## 2.6. API Catalogue.

Currently, the application uses default Laravel routes.

Example:
- GET / → Laravel Welcome Page

Future scope includes:
- REST APIs for marketing services
- CRUD operations for campaigns, users, analytics